

Project Proposal & System Structure

Project Name

Working name: Lebanon Local Commerce & Delivery Platform

This name is temporary. The final brand name can be decided later.

1. Executive Summary

The project is a connected mobile and web platform for Lebanon that links **clients, stores, service providers**, and **delivery drivers** through one shared backend system.

The platform is not only a delivery app. It is a complete local commerce and operations system where stores can manage products, inventory, orders, receipts, and deliveries, while clients can order products, request services, and track delivery from one place.

The platform will support different types of businesses, including restaurants, supermarkets, pharmacies, retail shops, laundry providers, repair shops, technicians, and other service providers.

The four main systems are:

1. **Client System**
2. **Store System**
3. **Service Provider System**
4. **Delivery System**

All four systems will have their own authorization, screens, permissions, workflows, and dashboards, but they will be connected through one backend and database.

An additional **Admin System** is required to manage the full platform.

2. Main Goal of the Platform

The main goal is to create a flexible platform where:

- Clients can order from stores.
- Clients can request services from providers.
- Stores can manage inventory, products, orders, and receipts.
- Service providers can manage requests, quotes, jobs, and service receipts.
- Delivery drivers can receive, accept, and complete delivery jobs.

- Stores can use their own drivers or request independent drivers.
- Admins can monitor and control the entire operation.

The system should work for Lebanon's local business environment, where many stores already have their own delivery drivers, many customers pay cash on delivery, and many services require phone/WhatsApp communication in the early stages.

3. Platform Type

This platform is best described as a:

Multi-sided local commerce, service, inventory, receipt, and delivery management platform.

Simpler description:

A platform that connects clients, stores, service providers, and delivery drivers, while allowing each business to manage its orders, inventory, receipts, and delivery operations.

4. Main Systems

4.1 Client System

The Client System is the customer-facing part of the app.

Clients can:

- Create an account.
- Log in.
- Add addresses and delivery locations.
- Browse stores.
- Browse restaurants.
- Browse service providers.
- Search for products and services.
- Place product orders.
- Request services.
- Request custom pickup and delivery.
- Track orders and deliveries.
- View receipts.
- Contact store, provider, driver, or support.
- Rate stores, providers, and drivers.
- Report problems.

Main Client Request Types

1. Product order from store.
 2. Restaurant order.
 3. Service request.
 4. Pickup and delivery request.
 5. Repair/laundry pickup request.
 6. Custom delivery request.
-

4.2 Store System

The Store System is for any business that sells physical products or menu items.

Examples:

- Restaurant
- Mini market
- Supermarket
- Pharmacy
- Clothing store
- Electronics store
- Bakery
- Flower shop
- Pet shop
- Cosmetics store
- Phone shop
- Butcher
- Stationery shop

Stores can:

- Create and manage store profile.
- Add products or menu items.
- Manage categories.
- Upload images.
- Set prices.
- Manage stock quantity.
- Set product availability.
- Receive orders.
- Accept or reject orders.
- Prepare orders.
- Mark orders as ready.
- Generate receipts.
- View order history.
- View sales reports.
- Manage store staff.

- Manage store drivers.
- Choose delivery method.
- Request independent delivery.
- Communicate with clients and drivers.

Store Delivery Options

Each store can choose one of these delivery modes:

1. Store uses its own delivery drivers.
2. Store uses platform independent drivers.
3. Store supports both.
4. Store allows pickup only.

4.3 Service Provider System

The Service Provider System is for businesses or individuals that provide services instead of normal products.

Examples:

- Laundry
- Phone repair
- Car wash
- Mechanic
- Electrician
- Plumber
- Cleaner
- Beauty service
- Home maintenance
- Tutor
- Technician
- Tailor
- Document service

Service providers can:

- Create provider profile.
- Add services.
- Set pricing type.
- Set availability.
- Accept or reject service requests.
- Send custom quotes.
- Communicate with clients.
- Request pickup/delivery if needed.
- Manage active jobs.

- Mark job as in progress.
- Mark job as completed.
- Generate service receipts.
- View earnings and service history.

Service Pricing Types

Services may have different pricing styles:

1. Fixed price.
2. Starting from price.
3. Hourly price.
4. Price after inspection.
5. Custom quote.

Example:

- Phone repair: starting from \$30.
- Plumbing: price after inspection.
- Cleaning: hourly price.
- Car wash: fixed price.

4.4 Delivery System

The Delivery System handles all delivery operations.

Delivery can be done by:

1. Independent platform drivers.
2. Store-owned drivers.
3. Service-provider drivers.

Drivers can:

- Create account.
- Log in.
- Complete driver profile.
- Set available/unavailable status.
- View available delivery jobs.
- Accept or reject delivery jobs.
- View pickup location.
- View dropoff location.
- Navigate to pickup/dropoff.
- Update delivery status.
- Contact client, store, or provider.
- Collect cash if needed.

- Confirm payment collection.
- View delivery history.
- View earnings.

Delivery Types

The system should support:

1. Store to client.
2. Restaurant to client.
3. Service provider to client.
4. Client to service provider.
5. Client to store.
6. Custom pickup and dropoff.
7. Two-way delivery for laundry, repair, tailoring, or documents.

For the MVP, the recommended delivery types are:

1. Store to client.
2. Service provider to client.
3. Custom pickup and dropoff.

Multi-stop and two-way delivery can be added later.

5. Admin System

The Admin System is required even though the main product has four systems.

Admins should be able to:

- Manage all users.
- Manage clients.
- Manage stores.
- Manage service providers.
- Manage drivers.
- Approve or reject stores/providers/drivers.
- View all orders.
- View all service requests.
- View all deliveries.
- View all receipts.
- Manage disputes and issues.
- Control zones.
- Control delivery fees.
- Control commissions.
- Manage categories.
- View platform reports.

- Suspend users or businesses.
- Manage support tickets.

The Admin System should be a web dashboard, not a mobile-only app.

6. How the Systems Connect

The systems are connected through a shared backend.

Main Connection Flow

```
Client creates order/request
↓
Store or Service Provider receives it
↓
Store/Provider accepts or rejects it
↓
If accepted, delivery may be created
↓
Driver receives or accepts delivery job
↓
Driver picks up item/order
↓
Driver delivers to client
↓
Payment is confirmed
↓
Receipt is generated
↓
Order/request is completed
```

Main Shared Data

The systems share:

- Users
- Roles
- Permissions
- Stores
- Service providers
- Products
- Inventory
- Orders

- Order items
 - Service requests
 - Deliveries
 - Payments
 - Receipts
 - Messages
 - Notifications
 - Ratings
 - Support tickets
-

7. Authorization and Permissions

Each system must have its own authorization.

A user should only access what they are allowed to access.

Main Roles

1. Admin
2. Support Agent
3. Client
4. Store Owner
5. Store Staff
6. Store Driver
7. Service Provider Owner
8. Service Provider Staff
9. Independent Driver

Important Rule

One user can have more than one role.

Example:

A person can be:

- A client
- A store owner
- A driver

So the system should not force one account to only have one role.

Recommended design:

User Account

↓

Can have multiple roles

↓

User chooses active role after login

Example:

Continue as Client

Continue as Store Owner

Continue as Driver

8. Permission Examples

Client Permissions

Clients can:

- View stores and providers.
- Place orders.
- Request services.
- Track own orders.
- View own receipts.
- Contact related store/provider/driver.
- Rate completed orders.

Clients cannot:

- See store inventory management.
- See driver earnings.
- See other clients' orders.
- Edit prices.
- Access admin data.

Store Owner Permissions

Store owners can:

- Manage store profile.
- Manage products.
- Manage inventory.

- Manage orders.
- Manage receipts.
- Manage store staff.
- Manage store drivers.
- View sales reports.
- Request delivery.

Store owners cannot:

- Access another store's data.
 - Access unrelated service provider requests.
 - Access platform-wide admin controls.
-

Store Staff Permissions

Store staff can:

- View assigned store orders.
- Accept or prepare orders if allowed.
- Update inventory if allowed.
- Contact drivers/clients for active orders.

Store staff may not be allowed to:

- Change store financial settings.
 - Delete the store.
 - Manage owner-level permissions.
-

Store Driver Permissions

Store drivers can:

- View assigned deliveries from their store.
- Update delivery status.
- Contact client for active delivery.
- Confirm delivery/payment collection.

Store drivers cannot:

- Accept independent platform jobs unless they also have independent driver role.
 - Edit inventory.
 - Edit product prices.
 - See full store financial reports.
-

Service Provider Owner Permissions

Provider owners can:

- Manage provider profile.
- Add services.
- Manage pricing.
- Accept/reject requests.
- Send quotes.
- Manage service receipts.
- Request delivery.
- View provider reports.

Provider owners cannot:

- Access store inventory unless they also own a store account.
 - Access other providers' data.
-

Independent Driver Permissions

Independent drivers can:

- View available delivery jobs.
- Accept delivery jobs.
- Update delivery status.
- Contact client/store/provider for active deliveries.
- View own delivery history.
- View own earnings.

Independent drivers cannot:

- Edit orders.
 - Edit inventory.
 - See store financial reports.
 - See other drivers' earnings.
-

Admin Permissions

Admins can:

- Access and manage everything.
- Approve stores/providers/drivers.
- View all orders, deliveries, payments, and receipts.
- Handle issues and disputes.

- Manage platform settings.
-

9. Core Modules

The backend should be built using modules.

Recommended modules:

1. Authentication Module
 2. Authorization / Roles / Permissions Module
 3. User Module
 4. Client Module
 5. Store Module
 6. Product Module
 7. Inventory Module
 8. Order Module
 9. Receipt Module
 10. Payment Module
 11. Delivery Module
 12. Driver Module
 13. Service Provider Module
 14. Service Request Module
 15. Notification Module
 16. Messaging Module
 17. Rating & Review Module
 18. Support Ticket Module
 19. Admin Module
 20. Reports & Analytics Module
-

10. Inventory Management

Inventory is one of the most important parts of the Store System.

Stores should be able to:

- Add stock.
- Remove stock.
- Adjust stock manually.
- Track product quantity.
- Set low-stock alerts.
- Mark products as available/unavailable.
- Automatically reduce stock after order confirmation.
- Restore stock if an order is cancelled before completion.

- View stock movement history.

Product Example

Product Name: Pepsi 330ml
Category: Drinks
Price: \$0.75
Quantity: 120
Low Stock Alert: 15
Available: Yes

If the client orders 2 units:

Old quantity: 120
Order quantity: 2
New quantity: 118

Inventory Movement Types

1. Stock added
2. Stock removed
3. Order sale
4. Order cancelled
5. Manual adjustment
6. Damaged item
7. Returned item
8. Expired item

Inventory movements are important because they allow the store to know why stock changed.

11. Receipts

Receipts should be generated for orders, service requests, and custom deliveries.

Store Order Receipt Should Include

- Receipt number
- Order number
- Store name
- Client name
- Products ordered
- Quantity

- Item price
- Subtotal
- Delivery fee
- Discount
- Total
- Payment method
- Payment status
- Driver name if delivery exists
- Date and time

Example Store Receipt

Receipt #R-000394

Store: Hamra Mini Market

Client: Mohamad

Order: #ORD-000928

Items:

2x Pepsi 330ml \$1.50

1x Chips \$0.80

Subtotal: \$2.30

Delivery fee: \$1.50

Total: \$3.80

Payment: Cash on delivery

Status: Paid

Date: 2026-05-01

Service Receipt Should Include

- Receipt number
- Service request number
- Provider name
- Client name
- Service name
- Quote amount
- Delivery fee if needed
- Total
- Payment method
- Payment status
- Date and time

Example Service Receipt

Receipt #SR-000142

Provider: Beirut Phone Repair

Client: Mohamad

Service:

iPhone screen replacement	\$45
---------------------------	------

Pickup fee:	\$2
-------------	-----

Return delivery fee:	\$2
----------------------	-----

Total:	\$49
--------	------

Payment: Cash

Status: Paid

12. Payments

For the MVP, the recommended payment method is:

1. Cash on delivery

Later, the system can support:

1. Online card payments
2. Wallet balance
3. Store subscriptions
4. Driver wallet
5. Bank transfer settlement
6. Payment gateway integration

Cash on Delivery Flow

Client places order

↓

Driver delivers order

↓

Client pays cash

↓

Driver marks payment collected

↓

Receipt becomes paid



Store/Admin can see cash collection record

13. Main Order Statuses

Product Order Statuses

1. Pending
2. Accepted by store
3. Rejected by store
4. Preparing
5. Ready for pickup
6. Driver assigned
7. Picked up
8. On the way
9. Delivered
10. Completed
11. Cancelled
12. Issue reported

Service Request Statuses

1. Pending
2. Accepted
3. Rejected
4. Waiting for quote
5. Quote sent
6. Quote accepted
7. In progress
8. Ready
9. Delivery requested
10. Completed
11. Cancelled
12. Issue reported

Delivery Statuses

1. Waiting for driver
2. Driver assigned
3. Driver arrived at pickup
4. Picked up

5. On the way
 6. Arrived at destination
 7. Delivered
 8. Failed delivery
 9. Cancelled
-

14. Main Workflows

14.1 Client Orders from Store

```
Client chooses store
↓
Client adds products to cart
↓
Client places order
↓
Store receives order
↓
Store accepts order
↓
Inventory is reserved or reduced
↓
Store prepares order
↓
Delivery is assigned
↓
Driver picks up order
↓
Driver delivers to client
↓
Payment is confirmed
↓
Receipt is generated
↓
Order is completed
```

14.2 Store Uses Own Driver

```
Client places order
↓
Store accepts order
```

```
↓  
Store assigns internal driver  
↓  
Internal driver picks up order  
↓  
Internal driver delivers order  
↓  
Payment is confirmed  
↓  
Receipt is generated  
↓  
Order completed
```

14.3 Store Requests Independent Driver

```
Client places order  
↓  
Store accepts order  
↓  
Store requests delivery  
↓  
Nearby independent drivers are notified  
↓  
Driver accepts job  
↓  
Driver picks up order  
↓  
Driver delivers to client  
↓  
Payment is confirmed  
↓  
Receipt is generated  
↓  
Order completed
```

14.4 Client Requests Service

```
Client chooses service provider  
↓  
Client sends service request  
↓
```

```
Provider accepts or rejects request
↓
Provider sends quote if needed
↓
Client accepts quote
↓
Provider performs service
↓
Delivery is requested if needed
↓
Payment is confirmed
↓
Receipt is generated
↓
Service request completed
```

14.5 Custom Pickup and Delivery

```
Client creates pickup/dropoff request
↓
System calculates delivery fee
↓
Available drivers are notified
↓
Driver accepts job
↓
Driver picks up item
↓
Driver delivers item
↓
Payment is confirmed
↓
Receipt is generated
↓
Delivery completed
```

15. Recommended MVP Scope

The MVP should prove the main business flow without overbuilding everything.

MVP Client System

- Register/login
- Browse stores
- Browse service providers
- View products/services
- Add to cart
- Place order
- Request service
- Add address/location
- Track order status
- View receipt
- Contact store/provider/driver

MVP Store System

- Register/login
- Manage store profile
- Add/edit/delete products
- Manage basic inventory
- Accept/reject orders
- Mark order preparing
- Mark order ready
- Request independent delivery
- Assign store driver
- View receipts

MVP Service Provider System

- Register/login
- Manage provider profile
- Add/edit/delete services
- Accept/reject requests
- Send quote
- Mark service in progress/completed
- Request delivery if needed
- View receipts

MVP Delivery System

- Register/login
- Driver profile
- Available/unavailable status
- See available jobs
- Accept/reject delivery
- View pickup/dropoff
- Update delivery status

- Mark cash collected
- View delivery history

MVP Admin System

- Admin login
 - Manage users
 - Approve stores/providers/drivers
 - View orders
 - View deliveries
 - View receipts
 - Manage issues
 - Manage zones and fees
-

16. Suggested Application Structure

Recommended products/apps:

1. **Client Mobile App**
2. **Driver Mobile App**
3. **Store & Service Provider Dashboard**
4. **Admin Dashboard**

Optional later:

1. Store mobile app
2. Service provider mobile app
3. Store driver app mode
4. Advanced inventory dashboard

Why This Structure Is Better

- Clients need a simple mobile app.
 - Drivers need a mobile app because they work on the road.
 - Stores and providers need dashboard tools because inventory, receipts, and order management are easier on larger screens.
 - Admins need a web dashboard for full control.
-

17. Suggested Technical Architecture

Frontend

Client App

- Flutter or React Native
- Android first, iOS later if needed

Driver App

- Flutter or React Native
- Real-time location support
- Push notifications

Store/Provider Dashboard

- React / Next.js
- Responsive web dashboard

Admin Dashboard

- React / Next.js
 - Role-based dashboard
-

Backend

Recommended options:

1. Node.js with NestJS
2. Laravel
3. Django / FastAPI

Recommended choice for scalable structure:

NestJS + PostgreSQL

Why:

- Clean module structure
 - Good for role-based systems
 - Good API architecture
 - Works well with WebSockets
 - Strong TypeScript support
-

Database

Recommended database:

PostgreSQL

Why:

- Good for structured business data
- Good for orders, payments, inventory, receipts
- Supports relationships well
- Reliable for financial records

Optional supporting tools:

- Redis for live driver status, queues, and caching
 - Object storage for product images and documents
 - Firebase Cloud Messaging for push notifications
 - WebSockets for live tracking and order updates
-

18. Core Database Models

This is the first draft of the main database models. These will be expanded later.

18.1 User

```
users
- id
- full_name
- phone
- email
- password_hash
- status
- created_at
- updated_at
```

18.2 Role

```
roles
- id
```

- name
- description

18.3 User Role

```
user_roles
- id
- user_id
- role_id
- entity_type
- entity_id
```

`entity_type` can be:

- store
- service_provider
- driver
- admin
- client

This allows one user to have multiple roles.

18.4 Store

```
stores
- id
- owner_user_id
- name
- description
- category_id
- phone
- address
- latitude
- longitude
- delivery_mode
- status
- opening_hours
- created_at
- updated_at
```

18.5 Product

```
products
- id
- store_id
- category_id
- name
- description
- price
- image_url
- is_available
- created_at
- updated_at
```

18.6 Inventory

```
inventory
- id
- store_id
- product_id
- quantity
- low_stock_threshold
- updated_at
```

18.7 Inventory Movement

```
inventory_movements
- id
- store_id
- product_id
- movement_type
- quantity_change
- reason
- reference_type
- reference_id
- created_by_user_id
- created_at
```

18.8 Order

```
orders
- id
- client_user_id
- store_id
- status
- subtotal
- delivery_fee
- discount
- total
- payment_method
- payment_status
- delivery_required
- created_at
- updated_at
```

18.9 Order Item

```
order_items
- id
- order_id
- product_id
- product_name_snapshot
- quantity
- unit_price_snapshot
- total_price
```

Product name and price should be stored as snapshots because prices may change later.

18.10 Service Provider

```
service_providers
- id
- owner_user_id
- name
- description
- category_id
- phone
- address
- latitude
```

- longitude
- status
- opening_hours
- created_at
- updated_at

18.11 Service

- ```
services
```
- id
  - provider\_id
  - category\_id
  - name
  - description
  - pricing\_type
  - base\_price
  - is\_available
  - created\_at
  - updated\_at

---

## 18.12 Service Request

- ```
service_requests
```
- id
 - client_user_id
 - provider_id
 - service_id
 - status
 - description
 - quote_amount
 - final_amount
 - payment_method
 - payment_status
 - delivery_required
 - created_at
 - updated_at

18.13 Driver

```
drivers
- id
- user_id
- driver_type
- vehicle_type
- phone
- status
- is_available
- current_latitude
- current_longitude
- created_at
- updated_at
```

`driver_type` can be:

- independent
- store_driver
- provider_driver

18.14 Delivery

```
deliveries
- id
- delivery_type
- order_id
- service_request_id
- driver_id
- pickup_name
- pickup_address
- pickup_latitude
- pickup_longitude
- dropoff_name
- dropoff_address
- dropoff_latitude
- dropoff_longitude
- status
- fee
- distance
- payment_collection_required
- cash_collected_amount
```

- created_at
- updated_at

`delivery_type` can be:

- store_to_client
- provider_to_client
- client_to_provider
- custom_pickup_dropoff

18.15 Receipt

```
receipts
- id
- receipt_number
- receipt_type
- order_id
- service_request_id
- delivery_id
- client_user_id
- store_id
- provider_id
- subtotal
- delivery_fee
- discount
- total
- payment_method
- payment_status
- generated_at
```

18.16 Payment

```
payments
- id
- payment_reference
- payment_type
- order_id
- service_request_id
- delivery_id
- amount
- method
- status
```

- collected_by_driver_id
- paid_at
- created_at

18.17 Message

```
messages
- id
- conversation_id
- sender_user_id
- message_type
- body
- attachment_url
- created_at
```

18.18 Notification

```
notifications
- id
- user_id
- title
- body
- type
- reference_type
- reference_id
- read_at
- created_at
```

18.19 Rating

```
ratings
- id
- from_user_id
- to_user_id
- target_type
- target_id
- rating
```

- comment
- created_at

18.20 Support Ticket

```
support_tickets
- id
- user_id
- related_order_id
- related_delivery_id
- related_service_request_id
- subject
- description
- status
- priority
- created_at
- updated_at
```

19. Suggested Folder Structure

This is a possible backend folder structure if using NestJS.

```
backend/
  src/
    main.ts
    app.module.ts

    common/
      guards/
      decorators/
      filters/
      interceptors/
      pipes/
      utils/

    config/
      database.config.ts
      app.config.ts
      auth.config.ts

    modules/
```

```
auth/
  auth.module.ts
  auth.controller.ts
  auth.service.ts
  dto/

users/
  users.module.ts
  users.controller.ts
  users.service.ts
  entities/
  dto/

roles/
  roles.module.ts
  roles.controller.ts
  roles.service.ts
  entities/

stores/
  stores.module.ts
  stores.controller.ts
  stores.service.ts
  entities/
  dto/

products/
  products.module.ts
  products.controller.ts
  products.service.ts
  entities/
  dto/

inventory/
  inventory.module.ts
  inventory.controller.ts
  inventory.service.ts
  entities/
  dto/

orders/
  orders.module.ts
  orders.controller.ts
  orders.service.ts
  entities/
  dto/

service-providers/
```

```
service-providers.module.ts
service-providers.controller.ts
service-providers.service.ts
entities/
dto/

services/
  services.module.ts
  services.controller.ts
  services.service.ts
  entities/
  dto/

service-requests/
  service-requests.module.ts
  service-requests.controller.ts
  service-requests.service.ts
  entities/
  dto/

drivers/
  drivers.module.ts
  drivers.controller.ts
  drivers.service.ts
  entities/
  dto/

deliveries/
  deliveries.module.ts
  deliveries.controller.ts
  deliveries.service.ts
  entities/
  dto/

receipts/
  receipts.module.ts
  receipts.controller.ts
  receipts.service.ts
  entities/

payments/
  payments.module.ts
  payments.controller.ts
  payments.service.ts
  entities/

notifications/
  notifications.module.ts
```

```
    notifications.gateway.ts
    notifications.service.ts

  messages/
    messages.module.ts
    messages.gateway.ts
    messages.service.ts
    entities/

  ratings/
    ratings.module.ts
    ratings.controller.ts
    ratings.service.ts

  support/
    support.module.ts
    support.controller.ts
    support.service.ts

  admin/
    admin.module.ts
    admin.controller.ts
    admin.service.ts
```

20. Suggested Frontend Structure

20.1 Client Mobile App

```
client-app/
  src/
    auth/
    onboarding/
    home/
    stores/
    store-details/
    products/
    cart/
    checkout/
    orders/
    order-tracking/
    service-providers/
    service-details/
    service-request/
    custom-delivery/
```

```
receipts/  
profile/  
support/  
notifications/
```

20.2 Driver Mobile App

```
driver-app/  
  src/  
    auth/  
    onboarding/  
    availability/  
    jobs/  
    job-details/  
    active-delivery/  
    navigation/  
    earnings/  
    history/  
    profile/  
    support/  
    notifications/
```

20.3 Store / Provider Dashboard

```
business-dashboard/  
  src/  
    auth/  
    layout/  
    dashboard/  
    store-profile/  
    provider-profile/  
    products/  
    inventory/  
    services/  
    orders/  
    service-requests/  
    deliveries/  
    receipts/  
    staff/  
    reports/
```

```
settings/  
support/
```

20.4 Admin Dashboard

```
admin-dashboard/  
  src/  
    auth/  
    layout/  
    overview/  
    users/  
    clients/  
    stores/  
    providers/  
    drivers/  
    orders/  
    service-requests/  
    deliveries/  
    receipts/  
    payments/  
    support-tickets/  
    zones/  
    categories/  
    commissions/  
    reports/  
    settings/
```

21. Design Phase: What Comes Next

After this proposal and structure file, the next step is the app design structure.

The next document should define:

1. What pages each system should have.
 2. What each page contains.
 3. What actions are available on each page.
 4. What data models each page uses.
 5. What API endpoints each page needs.
 6. What user permissions apply to each page.
 7. What the full user flows look like.
-

22. Next Design Document Outline

The next file should be:

App Pages, Models & Flow Design

It should include:

Client App Pages

- Splash screen
- Onboarding
- Login
- Register
- Role selection if needed
- Home
- Store listing
- Store details
- Product details
- Cart
- Checkout
- Order tracking
- Order history
- Receipt details
- Service provider listing
- Service provider details
- Service request form
- Custom delivery request
- Notifications
- Profile
- Addresses
- Support

Store System Pages

- Login
- Store dashboard
- Orders
- Order details
- Products
- Add/edit product
- Inventory
- Inventory movement history
- Receipts
- Store drivers
- Staff management

- Delivery settings
- Reports
- Store settings

Service Provider Pages

- Login
- Provider dashboard
- Service requests
- Request details
- Services
- Add/edit service
- Quotes
- Delivery requests
- Receipts
- Staff management
- Reports
- Provider settings

Delivery System Pages

- Login
- Driver onboarding
- Availability screen
- Available jobs
- Job details
- Active delivery
- Map/navigation
- Cash collection
- Delivery history
- Earnings
- Profile
- Support

Admin Dashboard Pages

- Admin overview
- User management
- Store approval
- Provider approval
- Driver approval
- Orders management
- Delivery management
- Receipt management
- Payment management
- Support tickets
- Zones

- Categories
 - Fees and commissions
 - Reports
 - Settings
-

23. Development Phases

Phase 1: Planning and Structure

Deliverables:

- Project proposal
- System structure
- User roles and permissions
- Main workflows
- Main database models
- App/page map

Phase 2: UI/UX Design

Deliverables:

- Wireframes
- User flows
- Page layouts
- Design system
- Components
- Mobile app screens
- Dashboard screens

Phase 3: MVP Backend

Deliverables:

- Authentication
- Role-based authorization
- Store module
- Product module
- Inventory module
- Order module
- Delivery module
- Receipt module
- Service provider module
- Notification module

Phase 4: MVP Frontend

Deliverables:

- Client app
- Driver app
- Store/provider dashboard
- Admin dashboard

Phase 5: Testing

Deliverables:

- Functional testing
- Role permission testing
- Order flow testing
- Delivery flow testing
- Inventory testing
- Receipt testing
- Admin testing

Phase 6: Launch Preparation

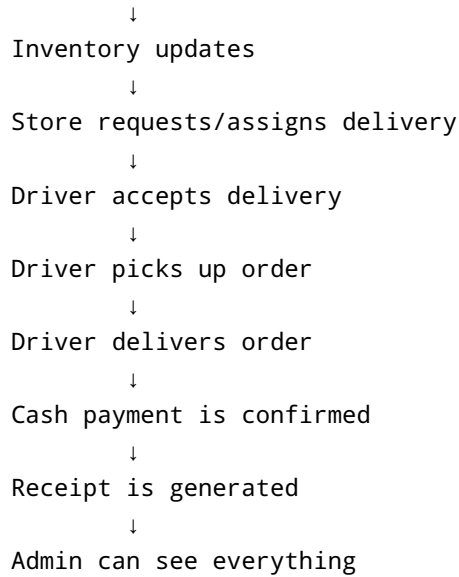
Deliverables:

- Deployment
- Admin accounts
- First stores/providers setup
- Driver onboarding
- Initial operating zones
- Support workflow

24. MVP Success Flow

The MVP should be considered successful when this flow works smoothly:

```
Client logs in
      ↓
Client browses store
      ↓
Client places order
      ↓
Store receives and accepts order
```



This flow proves the main platform idea.

25. Important Notes for Future Expansion

Later, the platform can add:

- Live driver tracking
- In-app chat
- WhatsApp integration
- Barcode scanning
- Advanced inventory reports
- Multi-store checkout
- Route optimization
- Driver wallet
- Store subscriptions
- Online payments
- Loyalty system
- Coupons and promotions
- AI-based product suggestions
- Demand forecasting
- Advanced analytics

These should not be built before the MVP is working.

26. Final Recommendation

The project should be built step by step.

Recommended first priority:

1. Define all pages and user flows.
2. Design the database properly.
3. Build authentication and authorization early.
4. Build store order flow.
5. Build delivery flow.
6. Build receipts and inventory updates.
7. Add service provider flow.
8. Add admin monitoring.

The most important foundation is:

One shared backend, multiple systems, strong role-based permissions, connected orders, inventory, deliveries, payments, and receipts.

This structure will allow the platform to grow without becoming messy later.